Computer Programming Using C(CPC)
DU # 2301CS101

Darshan UNIVERSITY
योग: कर्मसु कौशलम्

Unit-1
Overview of C





Prof. Vishal Kansagara
Computer Engineering Department
Darshan Institute of Engineering & Technology
Vishal.kansagara@darshan.ac.in
8200601076

Outline

- Algorithm and Flowchart
- Overview of C
- Header Files
- Structure of C Programs
- Data Types
- Input & Output Operation
- Token
- Operators in C
- Arithmetic expression & evaluation
- Math Header File

Algorithm and Flowchart

Darshan UNIVERSITY

Symbols used in flowchart



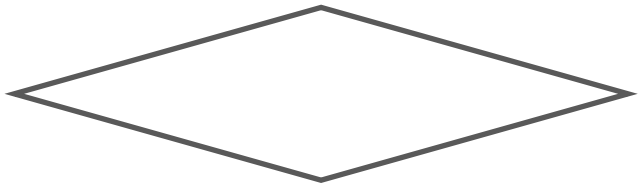
Start / Stop



Input / Output



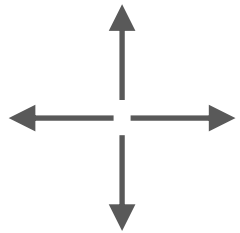
Process



Decision Making

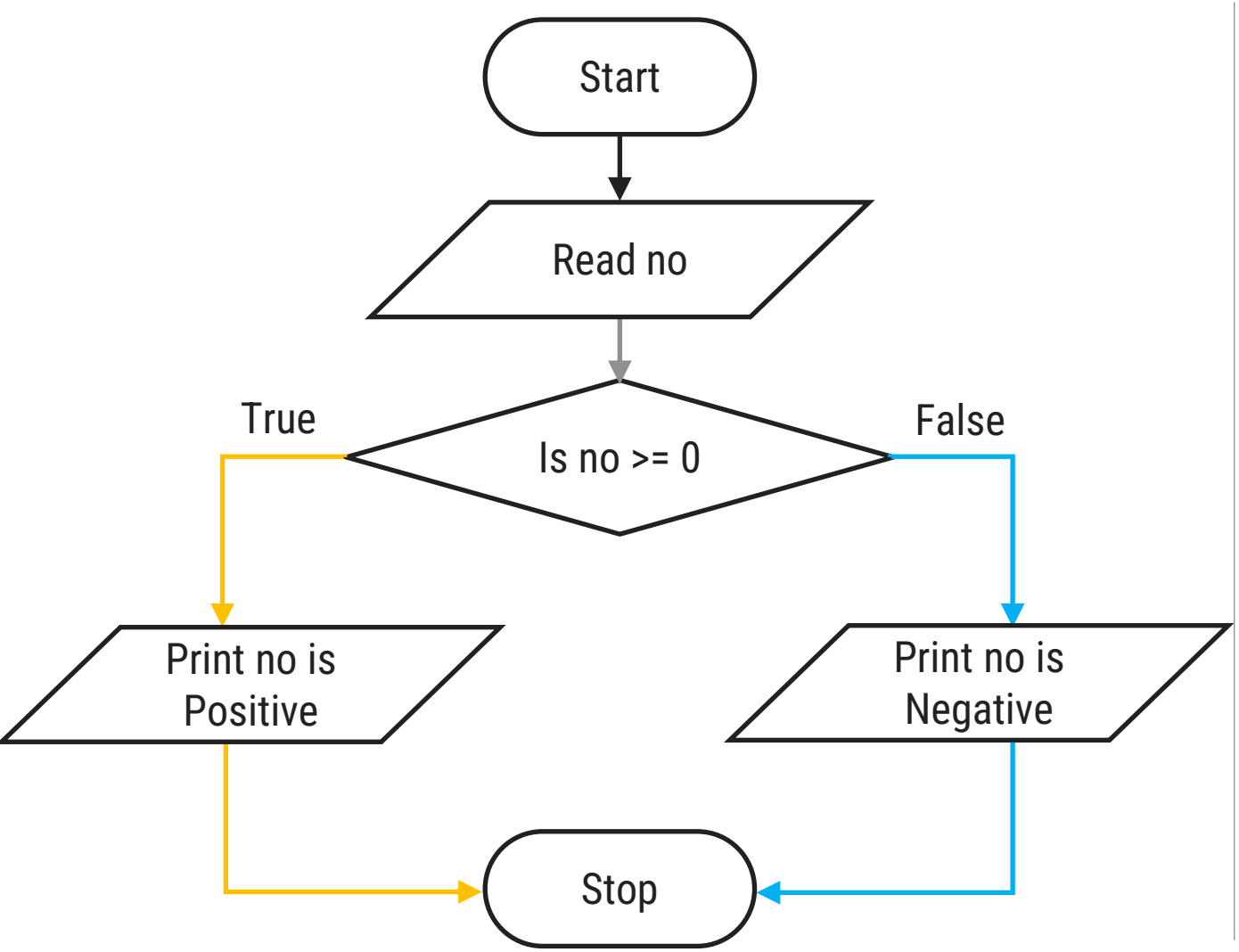


Subroutine



Arrows

Number is positive or negative (Flowchart and Algorithm)



Step 1: Read no.

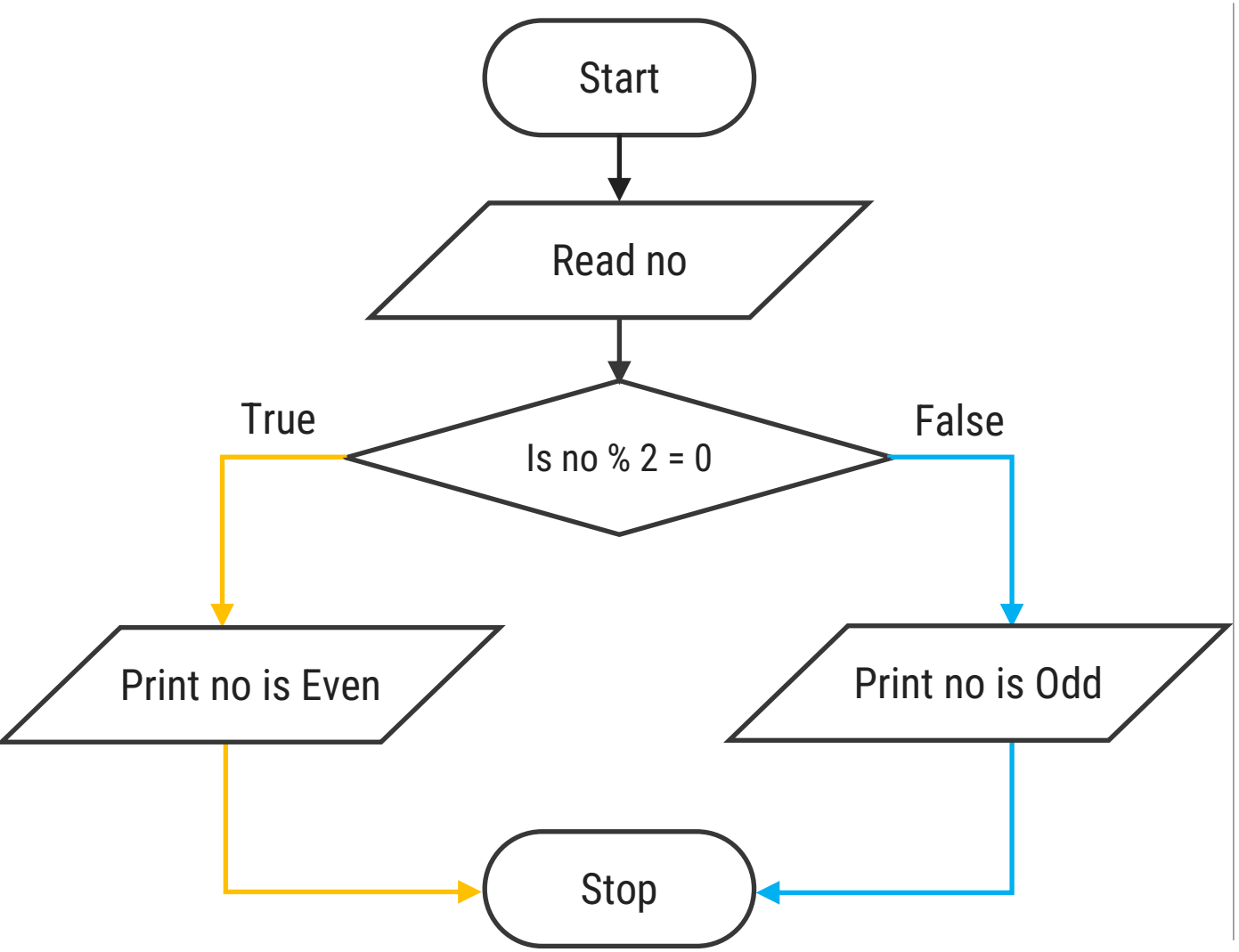
Step 2: If no is greater than equal zero, go to step 4.

Step 3: Print no is a negative number, go to step 5.

Step 4: Print no is a positive number.

Step 5: Stop.

Number is odd or even (Flowchart and Algorithm)



Step 1: Read no.

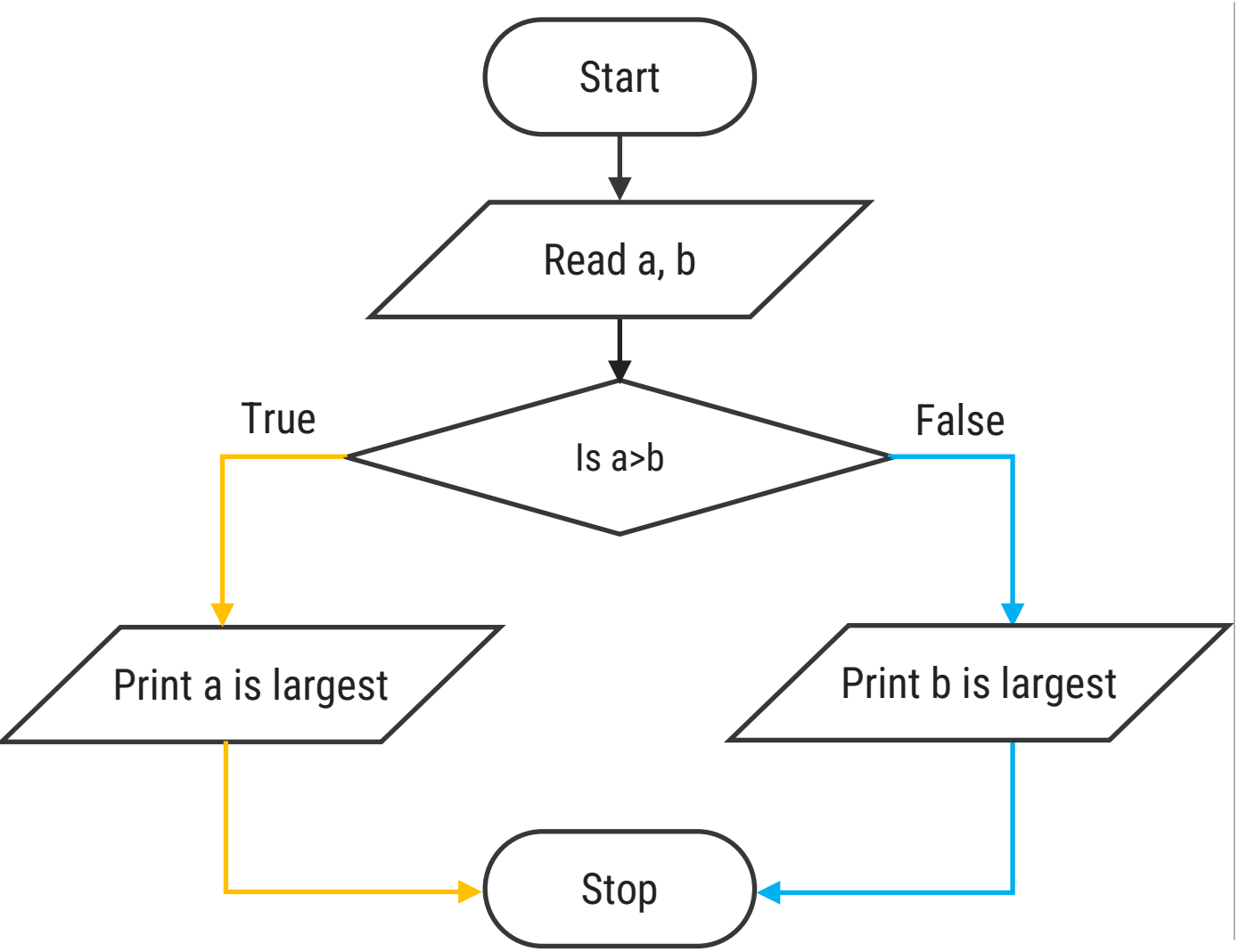
Step 2: If no mod 2 = 0,
go to step 4.

Step 3: Print no is a odd,
go to step 5.

Step 4: Print no is a even.

Step 5: Stop.

Largest number from 2 numbers (Flowchart and Algorithm)



Step 1: Read a, b.

Step 2: If a>b,

go to step 4.

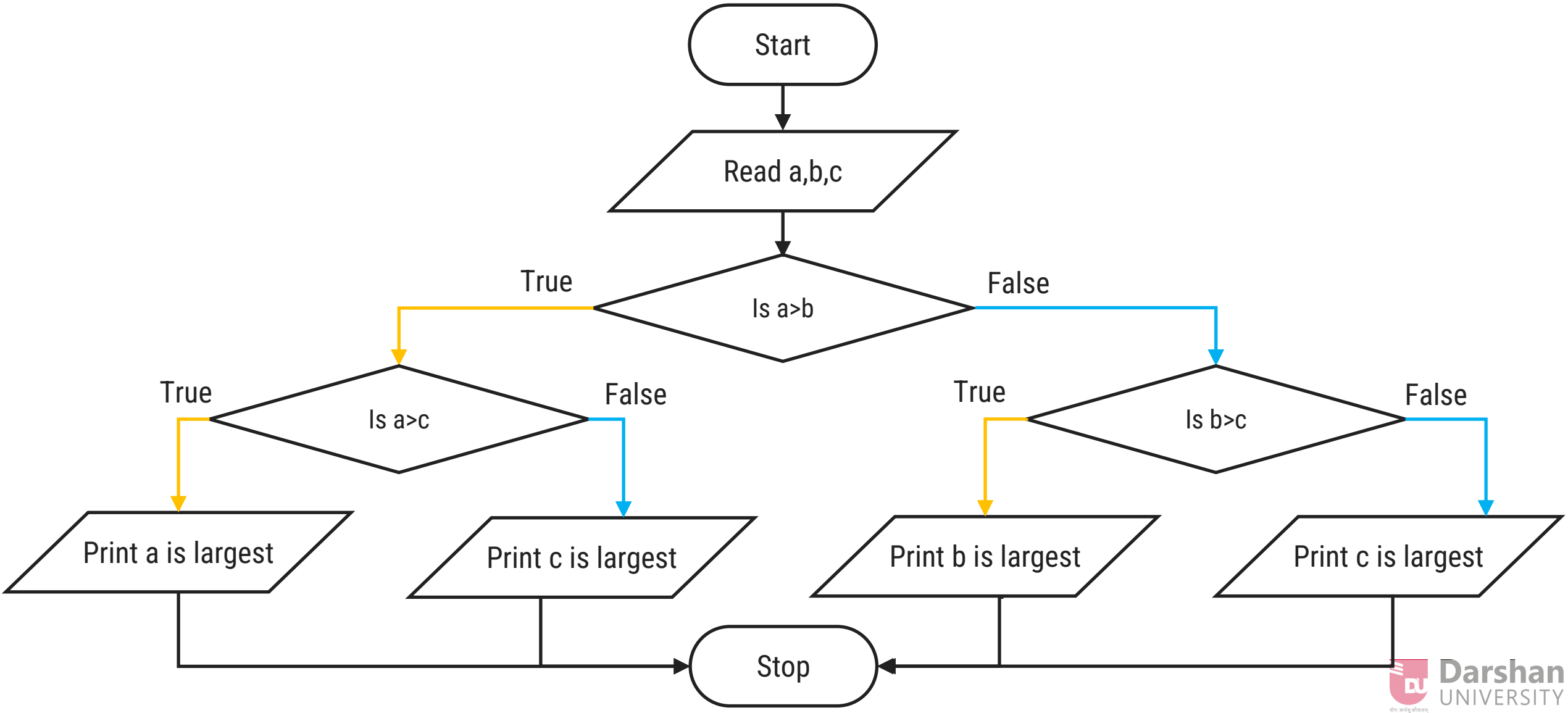
Step 3: Print b is largest number,

go to step 5.

Step 4: Print a is largest number.

Step 5: Stop.

Largest number from 3 numbers (Flowchart)



Largest number from 3 numbers (Algorithm)

Step 1: Read a, b, c.

Step 2: If a>b,

go to step 5.

Step 3: If b>c,

go to step 8.

Step 4: Print c is largest number,

go to step 9.

Step 5: If a>c,

go to step 7.

Step 6: Print c is largest number,

go to step 9.

Step 7: Print a is largest number,

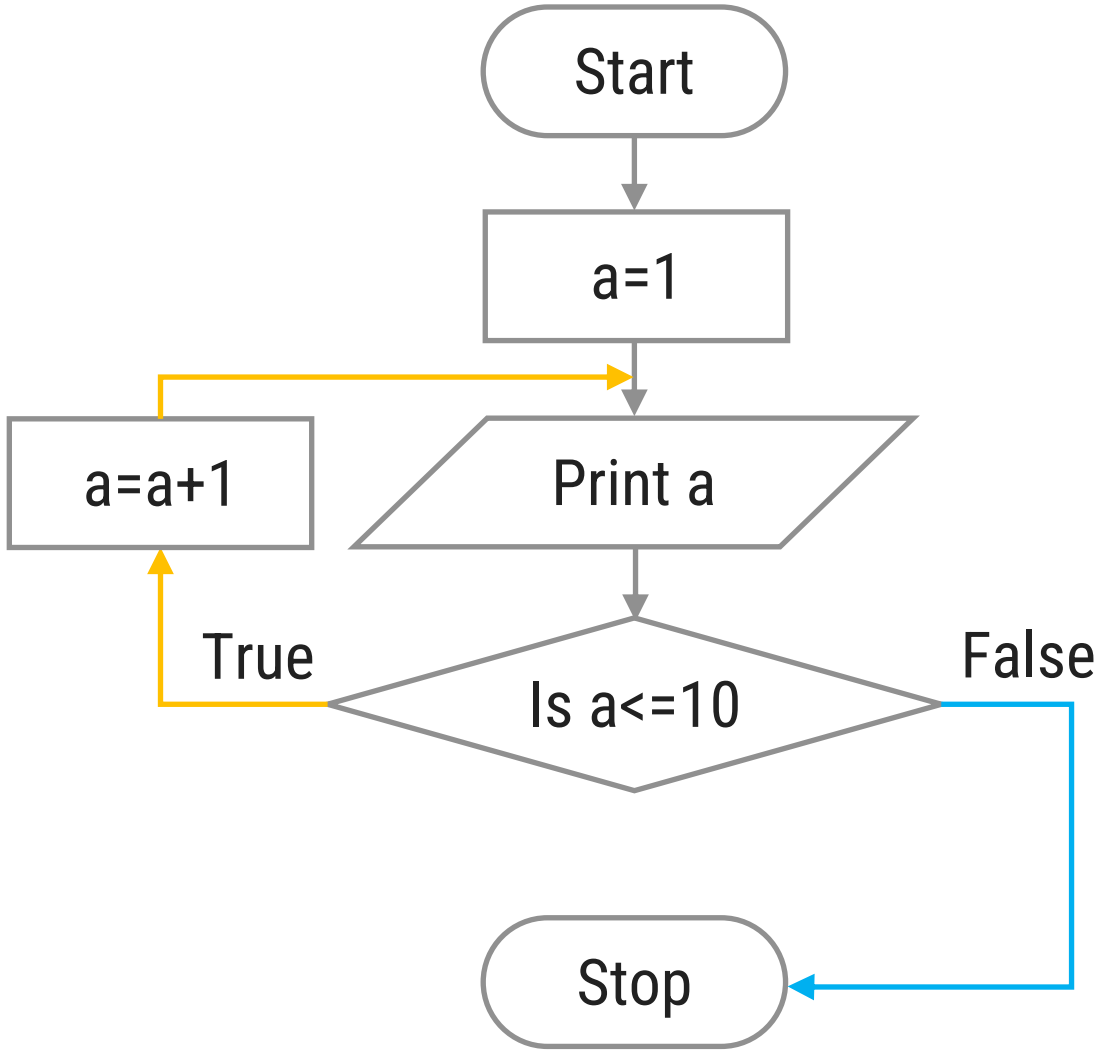
go to step 9.

Step 8: Print b is largest number.

Step 9: Stop.

Darshan UNIVERSITY

Print 1 to 10



Step 1: Initialize a to 1.

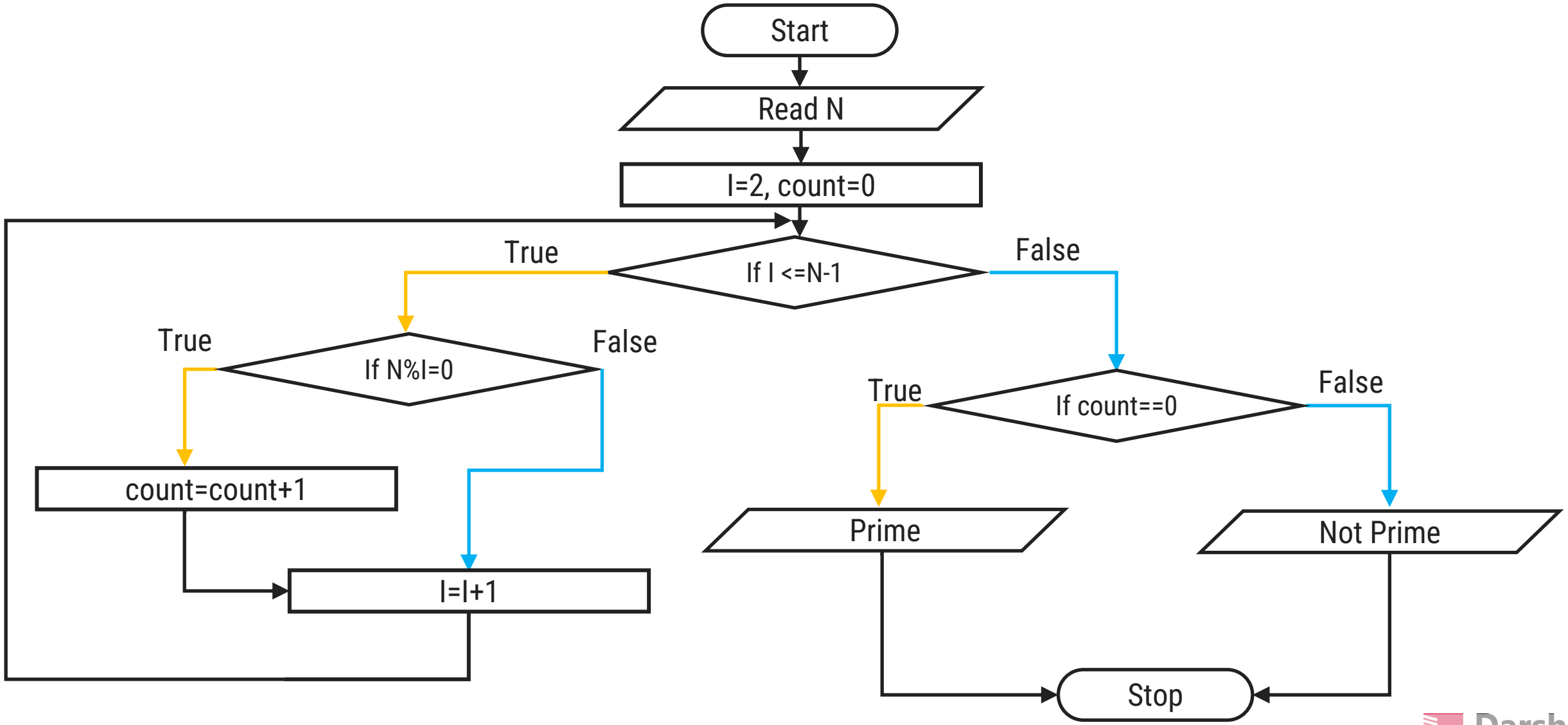
Step 2: Print a.

Step 3: Repeat step 2 until a<=10.

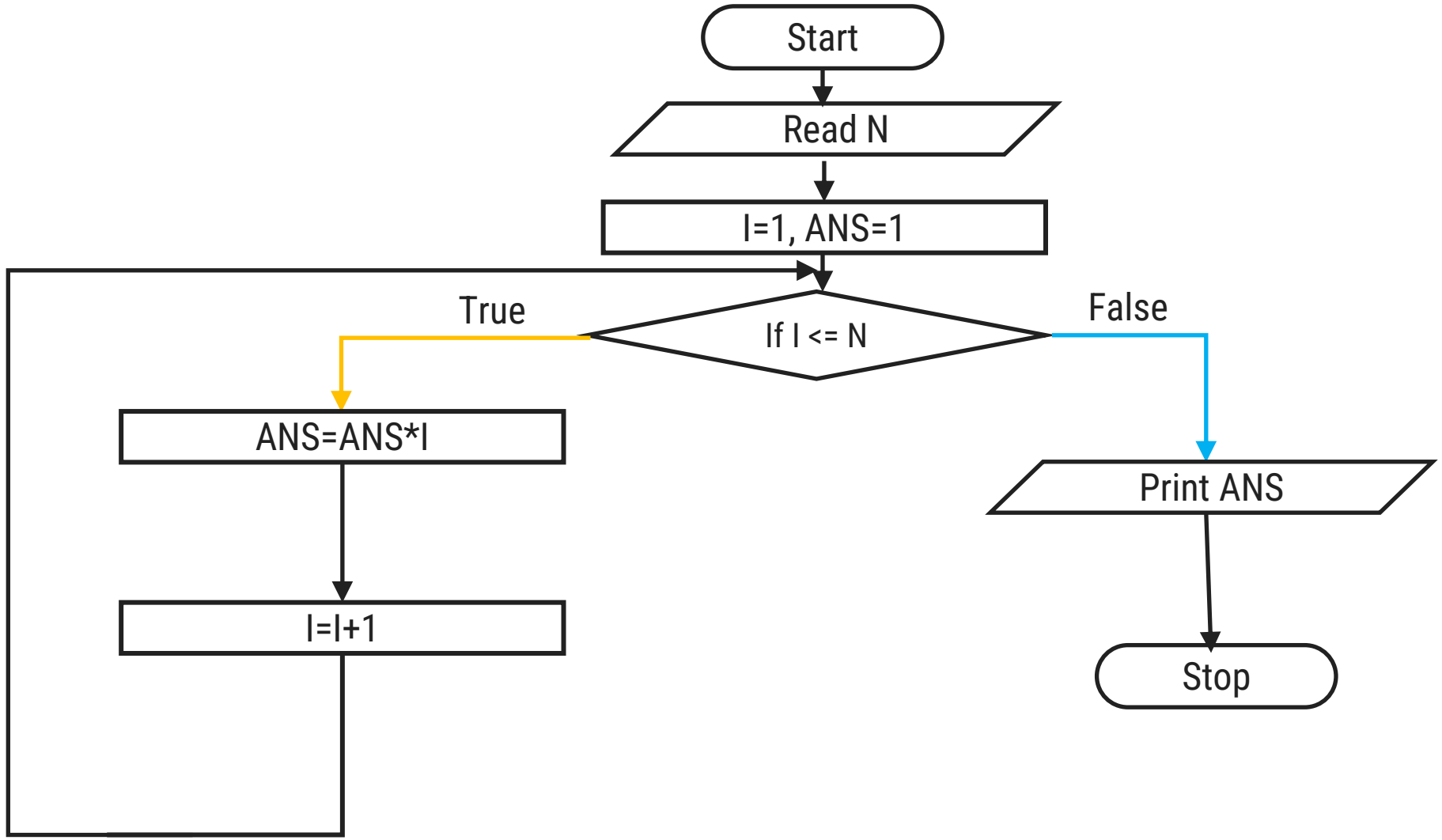
Step 3.1: a=a+1.

Step 4: Stop.

Prime Number (Flowchart)



Factorial (Flowchart)



C Programs

Darshan UNIVERSITY

FirstProg.c

```c
#include<stdio.h>

void main(){
    printf("Hello World");
}
```

Terminal

```
PS C:\> gcc FirstProg.c

PS C:\> ./a.exe

Hello World
```

SecondProg.c

```c
#include<stdio.h>

void main(){
     printf("Hello\n”);
     printf(“World”);
}
```

Terminal

```
PS C:\> gcc SecondProg.c

PS C:\> ./a.exe

Hello
World
```

History Of C

- C programming language was developed in 1972 by Dennis Ritchie at bell laboratories of AT&T (American Telephone & Telegraph).
- C language was developed to be used in UNIX operating system.
- C is widely used in development:
  - Operating Systems like Windows
  - Oracle database
  - Git
  - Python interpreter
  - etc.

Darshan UNIVERSITY

Why C Programming?

- Every programming language follows similar logic
- C builds problem-solving skills
- Base for Java, Python
- Used in Embedded Systems, OS and IoT

Darshan UNIVERSITY

ThirdProg.c

```c
#include<stdio.h>
void main(){
    int a=10,b=20,sum=0;
    sum=a+b;
    printf("%d",sum);
}
```

Terminal

```
PS C:\> gcc ThirdProg.c

PS C:\> ./a.exe
30
```

What is Variable and data type?

- Variable is a storage area which stores a value

  E.g. Placing a books in Bag, Where Bag is variable and book is Value

Type	Variable	Value
int	age	17
Float	percentage	88.25
char	grade	'A'
string	Name	“Vishal”

Datatypes

Data Type	Storage Space	Value Range	Example
Char	1 bytes	-128 to 127	‘a’, ’$’ ,’1’ , etc…
Unsigned char	1 bytes	0 to 255	‘a’, ’$’ ,’1’ , etc…
Int	4 bytes	-2,147,483,648 to 2,147,483,647	1, 2, 5, 9, 3, 0, etc..
Short int	2 bytes	-32,768 to 32,767	1, 2, 5, 9, 3, 0, etc..
Unsigned short int	2 bytes	O to 65,535	1, 2, 5, 9, 3, 0, etc..
long int	4 bytes	-2,147,483,648 to 2,147,483,647	1, 2, 5, 9, 3, 0, etc..
Unsigned long int	4 bytes	0 to 4,294,967,295	1, 2, 5, 9, 3, 0, etc..
Float	4 bytes	3.4E-38 to 3.4E+38	10.20, 20.30, 40.30, etc…
Double	8 bytes	1.7E-308 to 1.7E+308	10, 10.50, 100058, etc..
Long Double	16 bytes	3.4E-4932 to 1.1E+4932	10.20, 20.30, 40.30, etc…
Unsigned int	4 bytes	0 to 4,294,967,295	1, 2, 5, 9, 3, 0, etc..

- Note: Theoretically int is 2 bytes where as practically it is 4 bytes.

Darshan UNIVERSITY

FourthProg.c

```c
#include<stdio.h>
void main(){
    char ch='a';
    printf("%d %c",ch,ch);
}
```

```
Terminal
PS C:\> gcc FourthProg.c
PS C:\> ./a.exe
97 a
```

%d will print an ASCII value of character 'a' which is 97
%c will print a character 'a'

Note: In C, a char and its corresponding ASCII value can be used interchangeably in many contexts, because char is internally represented by an integer value (ASCII value).

FourthProg.c

```c
#include<stdio.h>
void main(){
    char ch=127;
    ch=ch+5;
    printf("%d",ch);
}
```

ch = 127 which is in binary

0111 1111 127

\+ 101 adding 5

1000 0100

Over here msb is 1 hence negative number is stored
to find out which number is store let's do 2's complement

1000 0100

0111 1011 1's Complement

1 add 1 to get 2's complement

0111 1010 which is 124 hence -124 is stored

Terminal

```
PS C:\> gcc FourthProg.c

PS C:\> ./a.exe

-124
```

Header Files

- C language has numerous libraries that include predefined functions to make programming easier.
- Header files contain the set of predefined standard library functions.
- Can use a header file in a program by including it with the C preprocessing directive #include.
- All the header file have .h extension.
- There are of 2 types of header file:
  - Pre-existing header files: Files which are already available in C compiler we just need to import them.
  - User-defined header files: These files are defined by the user as per need.
- We can include header files in our program by using one of the 2 below approach.

Header File

```
1  #include <filename.h>
2
3  Or
4
5  #include "filename.h"
```

Header Files

- #include<stdio.h>: It is used to perform input and output operations using functions scanf() and printf().
- #include<string.h>: It is used to perform various functionalities related to string manipulation like strlen(), strcmp(), strcpy(), size(), etc.
- #include<math.h>: It is used to perform mathematical operations like sqrt(), log2(), pow(), etc.
- #include<stdlib.h>: It is used for allocating and freeing memory at runtime (dynamic memory allocation/free)

Darshan UNIVERSITY

Importance Of C

- Modularity
- Extensibility
- Elegant syntax
- Case sensitive
- The standard library concept
- A powerful and varied range of operators

Escape Sequence

Sequence	Name	Description
\a	Alarm or Beep	It is used to generate a bell sound in the C program.
\n	New Line	It moves the cursor to the start of the next line.
\t	Horizontal Tab	It inserts some whitespace to the left of the cursor and moves the cursor accordingly.
\v	Vertical Tab	It is used to insert vertical space.
\\	Backlash	Use to insert backslash character.
\'	Single Quote	It is used to display a single quotation mark.
\"	Double Quote	It is used to display double quotation marks.
\b	Back Space	It is used to move the cursor backward.
\0	NULL	It represents the NLL character.

Practice printf()

- Print your name and address
- Print following pattern

```
* * * * *
*       *
*       *
*       *
* * * * *
```

- Print backslash

```
void main()
{
    printf("Hello \b\b\bHi Friends");
}
```

Structure of C programs

Darshan UNIVERSITY

Structure of C Program

- C program basically consists of the different parts
  - Preprocessor Commands
    - The first line of the program #include<stdio.h> is a preprocessor command, which tells a C compiler to include stdio.h file before going to actual compilation.
  - Functions
    - The *void main()* is the main function where the program execution begins.
  - Variables
    - A variable definition tells the compiler where and how much storage to create for the variable.
  - Statements & Expressions
    - A statement is a command given to the computer that instructs the computer to take a specific action, such as display to the screen, or collect input.
  - Comments
    - The line /*...*/ or // will be ignored by the compiler and it has been put to add additional comments in the program.

Structure

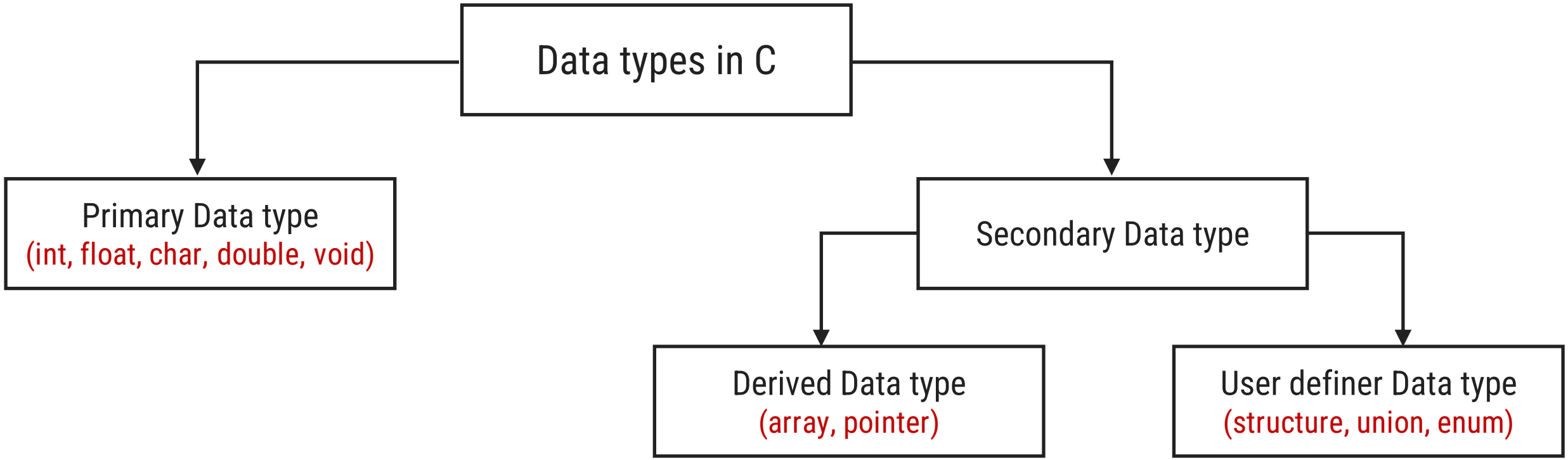
```
1  #include<stdio.h>
2  void main()
3  {
4     int variable_name;
5     printf("Enter two numbers to be added ");
6
7     // Comments
8  }
```

Data Types

Darshan UNIVERSITY

Data Types

- Data types are defined as the data storage format that a variable can store a data.
- It determines the type and size of data associated with variables.



Primary Data Type

- Primary data types are built in data types which are directly supported by machine.
- They are also known as fundamental data types.
  - int:
    - int datatype can store integer number which is whole number without fraction part such as 10, 105 etc.
    - C language has 3 classes of integer storage namely short int, int and long int. All of these data types have signed and unsigned forms.
    - Example: int a=10;
  - float:
    - float data type can store floating point number which represents a real number with decimal point and fractional part such as 10.50, 155.25 etc.
    - When the accuracy of the floating point number is insufficient, we can use the double to define the number. The double is same as float but with longer precision.
    - To extend the precision further we can use long double which consumes at least 10 Bytes of memory space. (generally 12 to 16 Bytes depends on compiler)
    - Example: float a=10.50;

Primary Data Type (cont…)

- char:
  - Char data type can store single character of alphabet or digit or special symbol such as ‘a’, ‘5’ etc.
  - Each character is assigned some integer value which is known as ASCII values.
  - Example: char a=‘a’;
- void:
  - The void type has no value therefore we cannot declare it as variable as we did in case of int or float or char.
  - The void data type is used to indicate that function is not returning anything.

Primary Data Type (cont…)

Data Type	Storage Space	Value Range	Example
Char	1 bytes	-128 to 127	‘a’, ’$’ ,’1’ , etc…
Unsigned char	1 bytes	0 to 255	‘a’, ’$’ ,’1’ , etc…
Int	4 bytes	-2,147,483,648 to 2,147,483,647	1, 2, 5, 9, 3, 0, etc..
Short int	2 bytes	-32,768 to 32,767	1, 2, 5, 9, 3, 0, etc..
Unsigned short int	2 bytes	O to 65,535	1, 2, 5, 9, 3, 0, etc..
long int	4 bytes	-2,147,483,648 to 2,147,483,647	1, 2, 5, 9, 3, 0, etc..
Unsigned long int	4 bytes	0 to 4,294,967,295	1, 2, 5, 9, 3, 0, etc..
Float	4 bytes	3.4E-38 to 3.4E+38	10.20, 20.30, 40.30, etc…
Double	8 bytes	1.7E-308 to 1.7E+308	10, 10.50, 100058, etc..
Long Double	16 bytes	3.4E-4932 to 1.1E+4932	10.20, 20.30, 40.30, etc…
Unsigned int	4 bytes	0 to 4,294,967,295	1, 2, 5, 9, 3, 0, etc..

- Note: Theoretically int is 2 bytes where as practically it is 4 bytes.

Darshan UNIVERSITY

Input & Output operation

Darshan UNIVERSITY

Input & Output Operations

- When we start dealing with variables in C need to do some input/output.
- The computer's primary input device is the keyboard, and its primary output device is the monitor.
- Input means to provide the data to be used in the program using variables.
- Output means to display data on the screen or write the data to a printer or a file.
- The functions used for standard input and output are present in the stdio.h header file.
- Functions used for input/output operations are:
  - printf()
  - scanf()

printf()

- printf() is a function defined in stdio.h file
- It displays output on standard output, mostly monitor
- Message and value of variable can be printed using this function
- Examples:
  - printf(“ ”);
  - printf(“Hello World”); // Hello World
  - printf(“%d”, c); // 15 suppose C=15
  - printf(“Sum = %d”, c); // Sum = 15
  - printf(“%d+%d=%d”, a, b, a+b); // 10+5=15

Darshan UNIVERSITY

scanf()

- scanf() is a function defined in stdio.h file
- scanf() function is used to read character, string, numeric data from keyboard
- Syntax of scanf
  - scanf("%X", &variable);
  - where %X is the format specifier which tells the compiler what type of data is in a variable.
  - & refers to address of variable which is directing the input value to a address returned by &variable.

scanf() (cont…)

Format Specifier	Supported data types	Example	Description
%c	Char	scanf("%c", &c)	Accept character value such as a, f, j, W, Z etc
%c	Unsigned char	scanf("%c", &c)	Accept character value such as a, f, j, W, Z etc
%d	Int	scanf("%d", &a)	Accept integer value such as 1, 5, 25, 105 etc
%hd	Short int	scanf("%hd", &a)	Accept integer value such as 1, 5, 25, 105 etc
%hu	Unsigned short int	scanf("%hu", &a)	Accept integer value such as 1, 5, 25, 105 etc
%ld	long int	scanf("%ld", &a)	Accept integer value such as 1, 5, 25, 105 etc
%lu	Unsigned long int	scanf("%lu", &a)	Accept integer value such as 1, 5, 25, 105 etc
%f	Float	scanf("%f", &b)	Accept floating value such as 1.5, 15.20 etc
%lf	Double	scanf("%lf", &b)	Accept floating value such as 1.5, 15.20 etc
%Lf	Long Double	scanf("%Lf", &b)	Accept floating value such as 1.5, 15.20 etc
%s	String	scanf("%s", d)	Accept string value such as diet, india etc
%u	Unsigned int	scanf("%u", &a)	Accept integer value such as 1, 5, 25, 105 etc

Exercise

- What is the output when each of the statements is performed? Assume x=2 and y=3

I/O Operation

```
printf("%d",x);
printf("%d",x+x);
printf("x=");
printf("x=%d",x);
printf("%d=%d",x+y,y+x);
```

Output

```
2
4
x=
x=2
5=5
```

Exercise

- Write a program that calculates squares and cubes of the numbers 0 to 5 and print like below

Output

```
number      square      cube
0           0           0
1           1           1
2           4           8
3           9           27
4           16          64
5           25          125
```

Input & Output Operations

I/O Operation

```
1 #include <stdio.h>
2 void main()
3 {
4       int user_input;
5       printf("Please enter a number: ");
6       scanf("%d", &user_input);
7       printf("You entered: %d", user_input);
8 }
```

Output

```
Please enter a number: 10
You entered: 10
```

Tokens

- The smallest individual unit of a program is known as token.
- C has the following tokens:
  - Keywords:
    - C reserves a set of 32 words for its own use. These words are called keywords (or reserved words), and each of these keywords has a special meaning within the C language.
  - Identifiers
    - Identifiers are names that are given to various user defined program elements, such as variable, function and arrays.
  - Constants
    - Constants refer to fixed values that do not change during execution of program.
  - Strings
    - A string is a sequence of characters terminated with a null character \0.
  - Special Symbols
    - Symbols such as #, &, =, *, ; are used in C for some specific function are called as special symbols.
  - Operators
    - An operator is a symbol that tells the compiler to perform certain mathematical operation.

Darshan UNIVERSITY

Keywords

- Keywords are predefined, reserved words in C language which is associated with specific features.
- They have special meaning to the compilers.
- There are total 32 keywords in C.

auto	continue	double	for	int	signed	struct	void
break	do	else	if	long	static	switch	while
case	default	enum	goto	register	sizeof	typedef	volatile
char	const	extern	float	return	short	union	unsigned

Identifier

- An identifier is used for any variable, function, data definition, labels in program.
- In C language, an identifier is a combination of alphanumeric characters.
- First begin with a letter of the alphabet or an underline, and the remaining are an alphabet, digit, or the underline.
- Identifiers must be unique.

```
1  #include<stdio.h>
2
3  void main()
4  {
5        int student;
6        float marks;
7  }
```



```
1  #include<stdio.h>
2
3  void main()
4  {
5        int student;
6        float student;
7  }
```



Constants

- Constant is a fixed value which cannot be changed throughout the program.
- There are two simple ways in C to define constants.
  - Using #define (Macro) :

Syntax

```
#define variable_name Value
```

Program

```
1  #include <stdio.h>
2  #define value 10
3  void main()
4  {
5              int data;
6              data = value*value;
7              printf("value of data : %d",data);
8  }
```

  - Using const keyword

Syntax

```
const data_type  variable_name = value;
```

Program

```
1  #include <stdio.h>
2  void main()
3  {
4              const int  value = 10;
5              int data;
6              data =value*value;
7              printf("value of data : %d",value);
8  }
```

Exercise: Identifier

1) Darshan	Valid
2) A	Valid
3) Age	Valid
4) void	Reserved word
5) MAX-ENTRIES	Invalid
6) double	Reserved word
7) time	Valid
8) G	Valid
9) Sue's	Invalid
10) return	Reserved word

12) xyz123	Valid
13) part#2	Invalid
14) "char"	Invalid
15) #include	Invalid
16) This_is_a_	Valid
17) _xyz	Valid
18) 9xyz	Invalid
19) main	Standard identifier
20) mutable	Reserved word
21) double	Reserved word
22) max?out	Invalid

Darshan UNIVERSITY

Exercise:

Program

```
1  void main()
2  {
3          int student;
4          float marks;
5  }
```

Answer

12

Explanation

Keyword: void, int, float
Identifier: main, student, marks
Special symbols: (,), ;, {, }

Program

```
1  void main()
2  {
3          int a,b,c,res;
4          res = a+b*c;
5          printf(“result is : %d",res);
6  }
```

Answer

30

Explanation

Keyword: void, int
Identifier: main, a, b, c, res, printf
Special symbols: (,), ;, {, }, ,(comma)
Operator: +,*,=
String: “result is : %d”

Exercise: Match the following pairs

(a)	`\n`
(b)	`3.145`
(c)	`-6513`
(d)	`'D'`
(e)	`4.25e-3`
(f)	`main()`
(g)	`%f, %d, %c`
(h)	`;`
(i)	`Constant`
(j)	`Variable`
(k)	`&`
(l)	`printf()`
(m)	`scanf()`

(1)	Literal
(2)	Statement Terminator
(3)	Character Constant
(4)	Escape Sequence
(5)	Input Function
(6)	Function
(7)	Integer Constant
(8)	Address of Operator
(9)	Output Function
(10)	Format Specifier
(11)	Exponential form
(12)	Real Constant
(13)	Identifier

(a)	`\n`	(4)	Escape Sequence
(b)	`3.145`	(12)	Real Constant
(c)	`-6513`	(7)	Integer Constant
(d)	`'D'`	(3)	Character Constant
(e)	`4.25e-3`	(11)	Exponential form
(f)	`main()`	(6)	Function
(g)	`%f, %d, %c`	(10)	Format Specifier
(h)	`;`	(2)	Statement Terminator
(i)	`Constant`	(1)	Literal
(j)	`Variable`	(13)	Identifier
(k)	`&`	(8)	Address of Operator
(l)	`printf()`	(9)	Output Function
(m)	`scanf()`	(5)	input Function

Operators in C

Darshan UNIVERSITY

Operators in C

- Arithmetic operators (+, - , *, /, %)
- Relational operators (<, <=, >, >=, ==, !=)
- Logical operators (&&, ||, !)
- Assignment operators (+=, -=, *=, /=)
- Increment and decrement operators (++, --)
- Conditional operators (?:)
- Bitwise operators (&, |, ^, <<, >>)
- Special operators

Arithmetic Operators

- Arithmetic operators are used for mathematical calculation.

Operator	Meaning	Example	Description
+	Addition	a + b	Addition of a and b
-	Subtraction	a – b	Subtraction of b from a
*	Multiplication	a * b	Multiplication of a and b
/	Division	a / b	Division of a by b
%	Modulo division- remainder	a % b	Modulo of a by b

Relational Operators

- Relational operators are used to compare two numbers and taking decisions based on their relation.
- Relational expressions are used in decision statements such as if, for, while, etc…

Operator	Meaning	Example	Description
<	Is less than	a < b	a is less than b
<=	Is less than or equal to	a <= b	a is less than or equal to b
>	Is greater than	a > b	a is greater than b
>=	Is greater than or equal to	a >= b	a is greater than or equal to b
==	Is equal to	a == b	a is equal to b
!=	Is not equal to	a != b	a is not equal to b

Logical Operators

- Logical operators are used to test more than one condition and make decisions.

Operator	Meaning
&&	logical AND (Both non zero then true, either is zero then false)
||	logical OR (Both zero then false, either is non zero then true)
!	logical NOT (It reverses the state of the answer)

a	b	a&&b	a||b
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

Assignment Operators

- Assignment operators (=) is used to assign the result of an expression to a variable.
- Assignment operator stores a value in memory.
- C also supports shorthand assignment operators which simplify operation with assignment.

Operator	Meaning
=	Assigns value of right side to left side
+=	a += 1 is same as a = a + 1
-=	a -= 1 is same as a = a - 1
*=	a *= 1 is same as a = a * 1
/=	a /= 1 is same as a = a / 1
%=	a %= 1 is same as a = a % 1

Increment and Decrement Operators

- Increment (++) operator used to increase the value of the variable by one.
- Decrement (--) operator used to decrease the value of the variable by one.

Example

```
x=100;
x++;
```

Explanation

After the execution the value of x will be 101.

Example

```
x=100;
x--;
```

Explanation

After the execution the value of x will be 99.

Increment and Decrement Operators (cont…)

Operator	Description
Pre increment operator (++x)	value of x is incremented before assigning it to the variable on the left

Example

```
x=10;
p=++x;
```

Explanation

First increment value of x by one then assign.

Output

x will be 11
p will be 11

Operator	Description
Post increment operator (x++)	First value of x is stored in temporary variable then incremented and after temporary variable value is assigning it to the variable on the left

Example

```
x=10;
p=x++;
```

```
temp = x;
x=x+1;
p = temp;
```

Explanation

First value of x is stored in temp variable then value of x is incremented and then value of temp variable is stored in p.

Output

x will be 11
p will be 10

Darshan UNIVERSITY

Conditional Operators

- A conditional operator is known as ternary operator.
- Syntax: *exp1 ? exp2 : exp3*

Working of the ? : Operator

exp1 is evaluated first
if exp1 is true(nonzero) then
- exp2 is evaluated and its value becomes the value of the expression

If exp1 is false(zero) then
- exp3 is evaluated and its value becomes the value of the expression

Example

```
m=2, n=3;
r=(m>n) ? m : n;
```

Explanation

Value of r will be 3

Example

```
m=2, n=3;
r=(m<n) ? m : n;
```

Explanation

Value of r will be 2

Exercise: What is output?

Example

```
void main()
{
    int a=0;
    a = (5>2) ? 10 : 8;
    printf("%d",a);
}
```

Example

```
void main()
{
    int a=0;
    a=5>2 ? printf("4"): printf("3");
}
```

Example

```
void main()
{
    int k = 8, m = 7, result;
    result = k > m ? k++ : m + 4;
    printf("%d,%d", result,k);
}
```

Bitwise Operators

- Bitwise operators are used to perform operation bit by bit.
- Bitwise operators may not be applied to float or double.

Operator	Meaning
&	bitwise AND
|	bitwise OR
^	bitwise exclusive OR
<<	shift left (shift left means multiply by 2)
>>	shift right (shift right means divide by 2)

Bitwise Operators

8 = 1000 (In Binary) and 6 = 0110 (In Binary)

Example: Bitwise & (AND)

```
int a=8, b=6, c;
c = a & b;
printf("Output = %d", c);
```

Output

0

Example: Bitwise | (OR)

```
int a=8, b=6, c;
c = a | b;
printf("Output = %d", c);
```

Output

14

Example: Bitwise << (Shift Left)

```
int a=8, b;
b = a << 1;
printf("Output = %d", b);
```

Output

16 (multiplying a by a power of two)

Example: Bitwise >> (Shift Right)

```
int a=8, b;
b = a >> 1;
printf("Output = %d", b);
```

Output

4 (dividing a by a power of two)

Bitwise operators tricks

Example

```
void main()
{
    int a = 24;
    printf("24/2=>%d\n", (a >> 1));
    printf("24/4=>%d\n", (a >> 2));
    printf("24/8=>%d\n", (a >> 3));
}
```

Right shift (>>) operator is equivalent to division by 2

Example

```
void main()
{
    int a = 12;
    printf("12*2=>%d\n", (a << 1));
    printf("12*4=>%d\n", (a << 2));
    printf("12*8=>%d\n", (a << 3));
}
```

Left shift (<<) operator is equivalent to multiplication by 2

Darshan UNIVERSITY

Bitwise operators tricks

Example

```
void main()
{
    int n = 10;
    n & 1? printf("ODD number"): printf("EVEN number");
}
```

Use bitwise AND (&) operator to check even or odd number

Example

```
void main()
{
      int num1 = 10;
      int num2 = 10;
      num1^num2 ? printf("Not Equal"): printf("Equal");
}
```

Use bitwise XOR (^) operator to check equality of two numbers

Darshan UNIVERSITY

Special Operators

Operator	Meaning
&	Address operator, it is used to determine address of the variable.
*	Pointer operator, it is used to declare pointer variable and to get value from it.
,	Comma operator, It is used to link the related expressions together.
sizeof	It returns the number of bytes the operand occupies.
.	member selection operator, used in structure.
->	member selection operator, used in pointer to structure.

Exercise: What is output?

Example

```
void main(){
  int a=5,b=10,c=0;
  c=!(a++&&++b)||c++;
  printf("%d %d %d",a,b,c);
}
```

Example

```
void main(){
  int a=5,b=10,c=0;
  c=(a++&&++b)||c;
  printf("%d %d %d",a,b,c);
}
```

Example

```
void main(){
  int a=5;
  a+=(a,a+=15,a*10);
  printf("%d",a);
}
```

Example

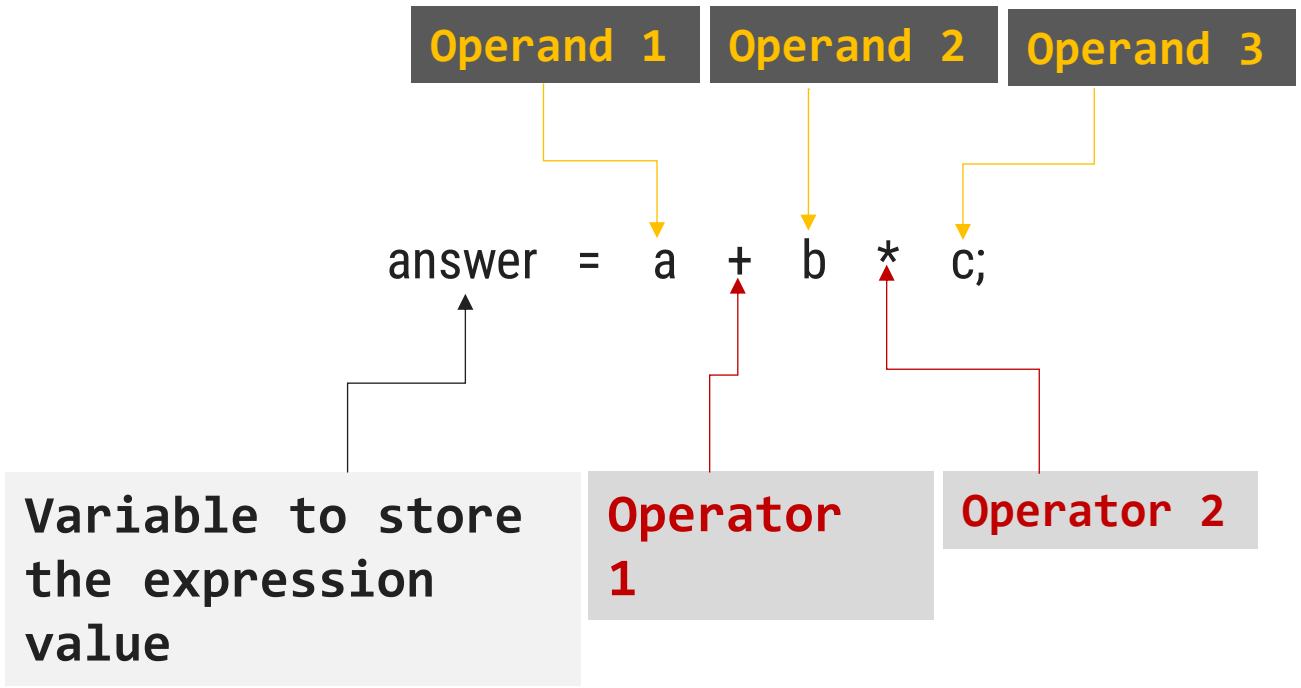
```
void main(){
  int a=5;
  a=a++;
  printf("%d",a);
}
```

Arithmetic Expressions & Evaluation

Darshan UNIVERSITY

Arithmetic Expressions

- An expression is a combination of operators, constants and variables.
- An expression may consist of one or more operands, and zero or more operators to produce a value.



Evaluation of Expressions

Operator	Type	Associativity
() [] . ->	Parentheses and selection	Left to Right
++ -- + - ! ~ (type) * & Sizeof	Unary Operator	Right to Left
* / %	Arithmetic operator	Left to Right
+ -	Arithmetic operator	Left to Right
<< >>	Shift operator	Left to Right
< <= > >=	Relational operator	Left to Right
== !=	Relational operator	Left to Right
&	Bitwise AND	Left to Right
^	Bitwise exclusive OR	Left to Right
|	Bitwise inclusive OR	Left to Right
&&	Logical AND	Left to Right
||	Logical OR	Left to Right
?:	Ternary conditional	Right to Left
= += -= *= /= %= &= ^=|= <<= >>=	Assignment operator	Right to Left
,	Comma operator	Left to Right

- When there are multiple operators in an expression, they are evaluated according to their precedence and associativity.
- While evaluating an arithmetic statement, some issues may cope up. For example, does the expression `2 * x – 3 * y` correspond to `(2x)-(3y)` or to `2(x-3y)`?
- An expression is evaluated based on the operator precedence and associativity.

Evaluation of Expressions

- An expression is evaluated based on the operator precedence and associativity.
- When there are multiple operators in an expression, they are evaluated according to their precedence and associativity.

Operator precedence

- Precedence of an operator is its priority in an expression for evaluation.
- The operator with higher precedence is evaluated first and the operator with the least precedence is evaluated last.
- Operator precedence is why the expression 5 + 3 * 2 is calculated as 5 + (3 * 2), giving 11, and not as (5 + 3) * 2, giving 16.
- We say that the multiplication operator (*) has higher "precedence" or "priority" than the addition operator (+), so the multiplication must be performed first.

Darshan UNIVERSITY

Operator associativity

- Associativity is the left-to-right or right-to-left order for grouping operands to operators that have the same precedence.
- Operator associativity is why the expression 8 - 3 - 2 is calculated as (8 - 3) - 2, giving 3, and not as 8 - (3 - 2), giving 7.
- We say that the subtraction operator (-) is "left associative", so the left subtraction must be performed first.
- When we can't decide by operator precedence alone in which order to calculate an expression, we must use associativity.

Exercise: Arithmetic Expression

- `x = 7 + 3 * 6 / 2 - 1;`
- `Output: 15`
- `x = 2 % 2 + 2 * 2 - 2 / 2;`
- `Output: 3`
- `X = 2 * 3 / 4 + 4 / 4 + 8 - 2 + 5 / 8;`
- `Output: 8`
- `x = ( 3 * 9 * ( 3 + ( 9 * 3 / ( 3 ) ) ) );`
- `Output: 324`
- `x = 10/5/2/1;`
- `Output: 1`
- `x = 2-3+5*2/8%3;`
- `Output: 0`

Exercise: Arithmetic Expression

Example

```
void main()
{
      int a=3,b=2,x=5,y=4,ans;
      ans = 5*b*b*x-3*a*y*y-8*b*b*x+10*a*y;
      printf("%d",ans);
}
```

Output

```
-84
```

Type Conversion

Darshan UNIVERSITY

Type conversion

- Type conversion is converting one type of data to another type.
- It is also known as Type Casting.
- There are two types of type conversion:
  - Implicit Type Conversion
    - This type of conversion is usually performed by the compiler when necessary without any commands by the user.
    - It is also called Automatic Type Conversion.
  - Explicit Type Conversion
    - These conversions are done explicitly by users using the pre-defined functions.

Example: Implicit Type Conversion

```
1  int a = 20;
2  double b = 20.5;
3  printf("%lf", a + b);
```

Output

```
40.500000
```

Example: Explicit Type Conversion

```
1  double a = 4.5, b = 4.6, c = 4.9;
2  int result = (int)a + (int)b + (int)c;
3  printf("result = %d", result);
```

Output

```
12
```

Formatted Output

Darshan UNIVERSITY

Formatted Output

- The format specification for printing an integer number is: *%w d*
- where w specifies the minimum field width for the output. However, if a number is greater than the specified field width, it will be printed in full.
- The number is written right justified in the given field width.
- It is possible to force the printing to be left-justified by placing a minus sign directly after the %character.

Formatted Output integers

Format	Output
printf("%d",9876)	9 8 7 6
printf("%6d",9876)	_ _ 9 8 7 6
printf("%d",9876)	9 8 7 6
printf("%-6d",9876)	9 8 7 6 _ _
printf("%06d",9876)	0 0 9 8 7 6

Formatted Output

Formatted Output

```
void main()
{
	int m = 12345;
	printf("%d\n",m);
	printf("%10d\n",m);
	printf("010%d\n",m);
	printf("%-10d\n",m);
	printf("%-10d\n",-m);
	printf("%10d\n",-m);
}
```

Output

```
12345
     12345
01012345
12345
-12345
    -12345
```

Formatted Output real/float numbers

```
float m = 98.7654
```

Format	Output
printf("%7.4f",m);	9 8 . 7 6 5 4
printf("%7.2f",m);	9 8 . 7 7
printf("%-7.2f",m);	9 8 . 7 7
printf("%f",m);	9 8 . 7 6 5 4

Math Functions

- C Programming allows us to perform mathematical operations through the functions defined in <math.h> header file.

Sr no.	Function	Description
1	ceil(number)	rounds up the given number & returns the integer value which is greater than or equal to given number.
2	floor(number)	rounds down the given number & returns the integer value which is less than or equal to given number.
3	sqrt(number)	returns the square root of given number.
4	pow(base, exponent)	returns the power of given number.
5	abs(number)	returns the absolute value of given number.

Math Functions

Math Function

```c
#include <stdio.h>
#include <math.h>
void main()
{
        printf(“Ceil: %f",ceil(3.6));
        printf("\nCeil: %f",ceil(3.3));
        printf("\nFloor: %f",floor(3.6));
        printf("\nFloor: %f",floor(3.2));
        printf("\nSquare Root: %f",sqrt(16));
        printf("\nSquare Root: %f",sqrt(7));
        printf("\nPower: %f",pow(2,4));
        printf("\nPower: %f",pow(3,3));
        printf("\nAbsolute: %d",abs(-12));
}
```

Output

```
Ceil: 4.000000
Ceil: 4.000000
Floor: 3.000000
Floor: 3.000000
Square Root: 4.000000
Square Root: 2.645751
Power: 16.000000
Power: 27.000000
Absolute: 12
```

Thank you

Darshan UNIVERSITY